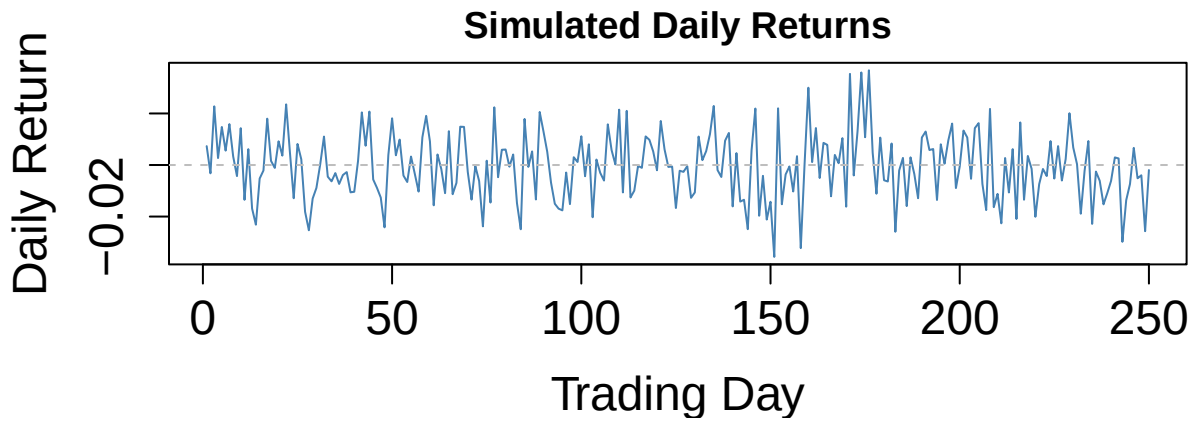
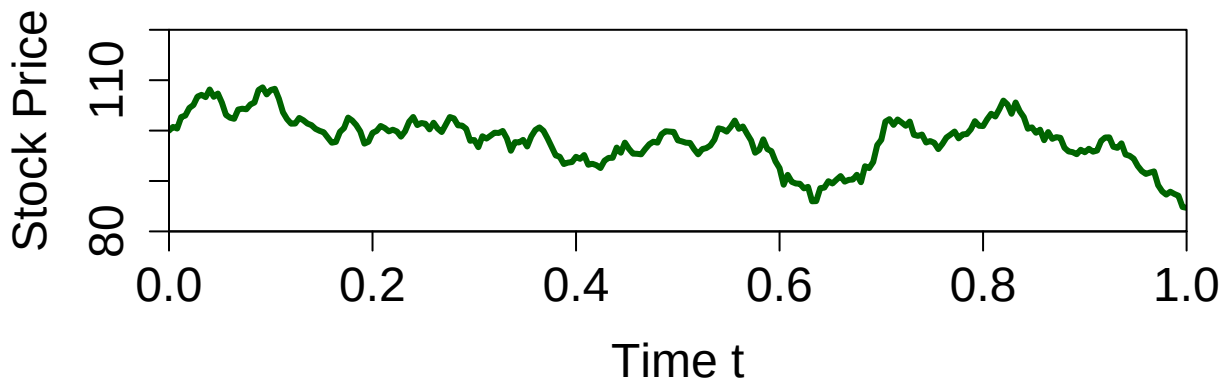
Exercise_1_Lama.R

adria

2026-03-09

```r
#####################################################
# Exercise 1
#####################################################

##################################################################################
# Exercise 1: Simulating a GBM
#---------------------------------------------------------------------------------
# adjust this accoridng to your working dir
#setwd("P:/HSG/SMFI/SMFI-main/problem_sets/Lama")

#-------------------------------------------------------------
# Exercise 1 a)
S0 <- 100                          # Current stock price
mu <- 0.08                         # Mean return rate 8%
sigma <- 0.2                       # Volatility of returns 20%
T <- 1                             # Price path over 1 year
s <- 250                           # Trading Days
delta_t <- rep(1/s,T*s)            # Daily time steps
t <- c(0,cumsum(delta_t))          # Time line including start date
paths <- 1                         # Paths
set.seed(13)                       # Random number generator

#-------------------------------------------------------------
delta_W <- sqrt(delta_t) * rnorm(n=T * s, mean = 0, sd=1) # Vector of path increments
Rt <- ((mu * delta_t)+(sigma*delta_W))                    # Return path

St <- c(S0, S0*cumprod(1+Rt))                             # Convert into price path



par(mfrow = c(2, 1), mar = c(4, 5, 2, 1))

plot(1:s, Rt, type = "l", col = "steelblue", lwd = 1,
     xlab = "Trading Day", ylab = "Daily Return",
     cex.axis = 1.5, cex.lab = 1.5,
     main = "Simulated Daily Returns")
abline(h = 0, lty = 2, col = "grey")

plot(t, St, type = "l", col = "darkgreen", lwd = 3,
     xlab = "Time t", ylab = "Stock Price",
     cex.axis = 1.5, cex.lab = 1.5, cex.main = 1.5,
     xaxs = "i", yaxs = "i", ylim = c(80, 120))
```






```r
par(mfrow = c(1, 1))

##############################################################################
# Exercise 1 b)
#-----------------------------------------------------------------------------
dar <- mean(Rt)                # Average daily returns
annualized_mu <- 250*dar       # Annulized Returns
print(annualized_mu)
```

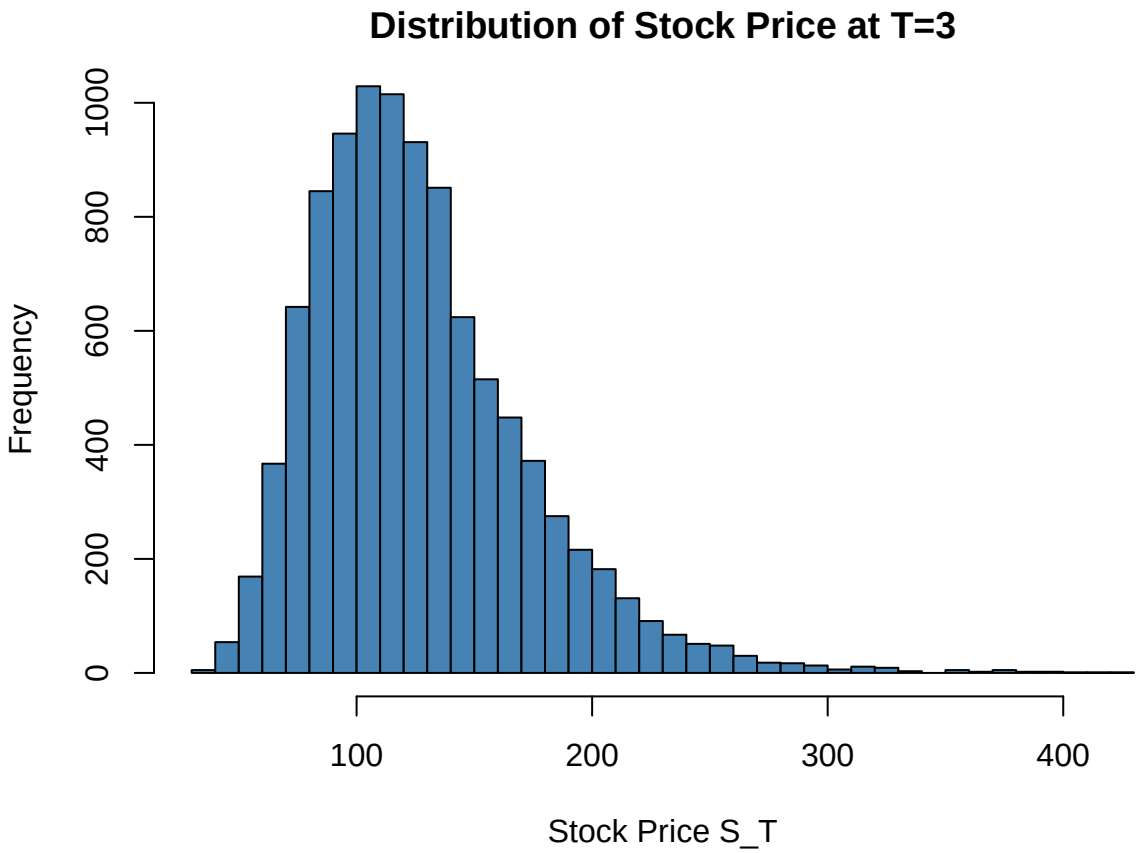
```
## [1] -0.1456746
```

```r
# Report: The annualized sample mean differs from the true mu = 8% due to
# sampling variability. A single path consists of only 250 daily returns,
# which is a finite sample. The sample mean is an estimator of mu,
# but any single realization will deviate from the true value due to randomness.
# This deviation shrinks as the sample size increases.

##############################################################################
# Exercise 1 c)
#-----------------------------------------------------------------------------
N <- 10000       # Number of paths
set.seed(13)

# Generate a Matrix
delta_W_matrix <- sqrt(1/s) * matrix(rnorm(N * s, mean = 0, sd = 1), nrow = N, ncol = s)
```

```r
# Calculate Returns for all paths
Rt_matrix <- (mu * (1/s)) + (sigma * delta_W_matrix)

# Calculate the mean
dar_MC <- mean(Rt_matrix)

# Annulize the mean
annualized_mu_MC <- 250 * dar_MC

# Print the mean
print(annualized_mu_MC)
```

```
## [1] 0.08104439
```

```r
# Report: The Mean for 10'000 Paths is much closer to the true mu.
# It's because of the law of large numbers. It has to approx to 8%.

################################################################################
# Exercise 1 d)
#-------------------------------------------------------------------------------
T <- 3                              # Maturity T = 3 years
S0 <- 100                           # Current stock price
mu <- 0.08                          # Mean return rate 8%
sigma <- 0.2                        # Volatility of returns 20%
paths <- 10000
set.seed(13)

# Simulate W_T. It's normally distributed with mean 0 and variance T
W_T <- sqrt(T) * rnorm(paths, mean = 0, sd = 1)

# Compute 10'000 terminal prices using exact solution
S_T <- S0 * exp((mu - (sigma^2) / 2) * T + sigma * W_T)

# Plot Histogram
hist(S_T, breaks = 50, col = "steelblue",
     main = "Distribution of Stock Price at T=3",
     xlab = "Stock Price S_T", ylab = "Frequency")
```




```r
# Compare sample average and expectation
sample_avg <- mean(S_T)
theo_exp <- S0 * exp(mu * T)

print(paste("Sample Average:", round(sample_avg, 4)))
```

```
## [1] "Sample Average: 126.8325"
```

```r
print(paste("Theoretical Expectation:", round(theo_exp, 4)))
```

```
## [1] "Theoretical Expectation: 127.1249"
```

```r
# Report:
# Shape: The histogram is right-skewed, which is expected because S_T follows
# a lognormal distribution. Prices cannot fall below zero but have no upper
# bound, producing a right tail.
#
# Comparison: The sample average is close to the theoretical expectation
# E[S_T] = S0 * exp(mu * T) due to the law of large numbers. With N = 10'000
# simulations, the randomness averages out. The small remaining difference
# is finite-sample noise, which would shrink further as N increases.
```

Exercise_2_Lama.R

adria

2026-03-09

```r
#####################################################
# Exercise 2: Pricing a Double Digital Option
#####################################################

# Exercise 2 a)
S0 <- 100                          # Current stock price
#mu <- 0.08                        # Mean return rate 8%
sigma <- 0.2                       # Volatility of returns 20%
r <- 0.015                         # Continuously compounded risk-free rate
T <- 2                             # Price path over 2 year
K1 <- 105                          # Lower Strike
K2 <- 115                          # Upper Strike
H <- 20000                         # Lump Sum Payoff
paths <- 100000                    # Number of Simulations
set.seed(13)                       # Set random number


# Simulate stock prices at maturity
#-------------------------------------------
# Simulate Wiener process levels at maturity
WT <- sqrt(T)*rnorm(paths, mean = 0, sd = 1)
# Simulate stock prices at maturity
ST <- S0*exp((r-sigma^2/2)*T + sigma*WT)



# Compute option payoffs
#-------------------------------------------
# The indicator function: 1 if K1 <= S_T <= K2, else 0
DT <- H * as.numeric(ST >= K1 & ST <= K2)


# Discount expected payoff to get the option price
#-------------------------------------------
V0 <- exp(-r * T) * mean(DT)


# Report result
#-------------------------------------------
cat("Double Digital Option Price (V0):", round(V0, 2), "\n")
```

```
## Double Digital Option Price (V0): 2352.94
```

```r
cat("Average payoff E^Q[D_T]:", round(mean(DT), 2), "\n")
```

```
## Average payoff E^Q[D_T]: 2424.6
```

```r
cat("Fraction of paths in [K1, K2]:", round(mean(ST >= K1 & ST <= K2), 4), "\n")
```

```
## Fraction of paths in [K1, K2]: 0.1212
```

```r
# Report: The simulated price of the double digital option is V0 = 2352.94
# (Value depends on seed). This reflects the discounted expected payoff under
# the risk-neutral measure Q.
```

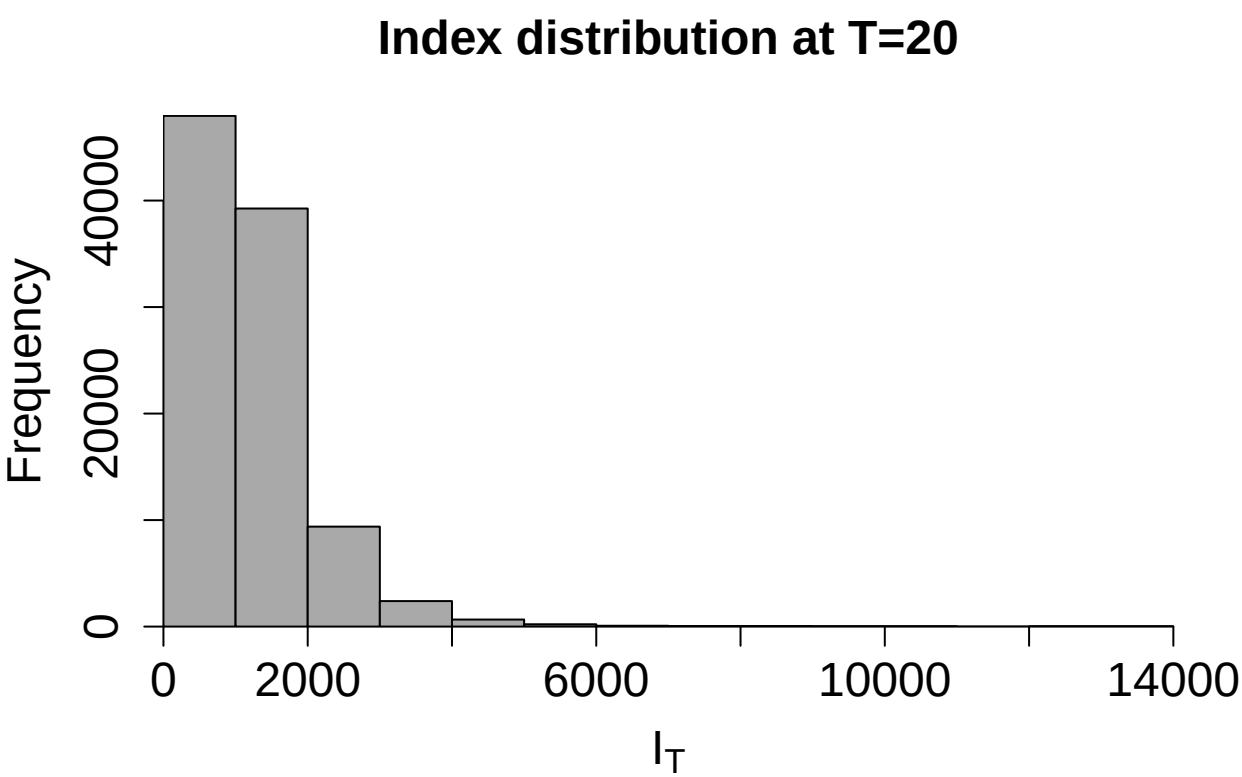
Exercise_3_Lama.R

adria

2026-03-09

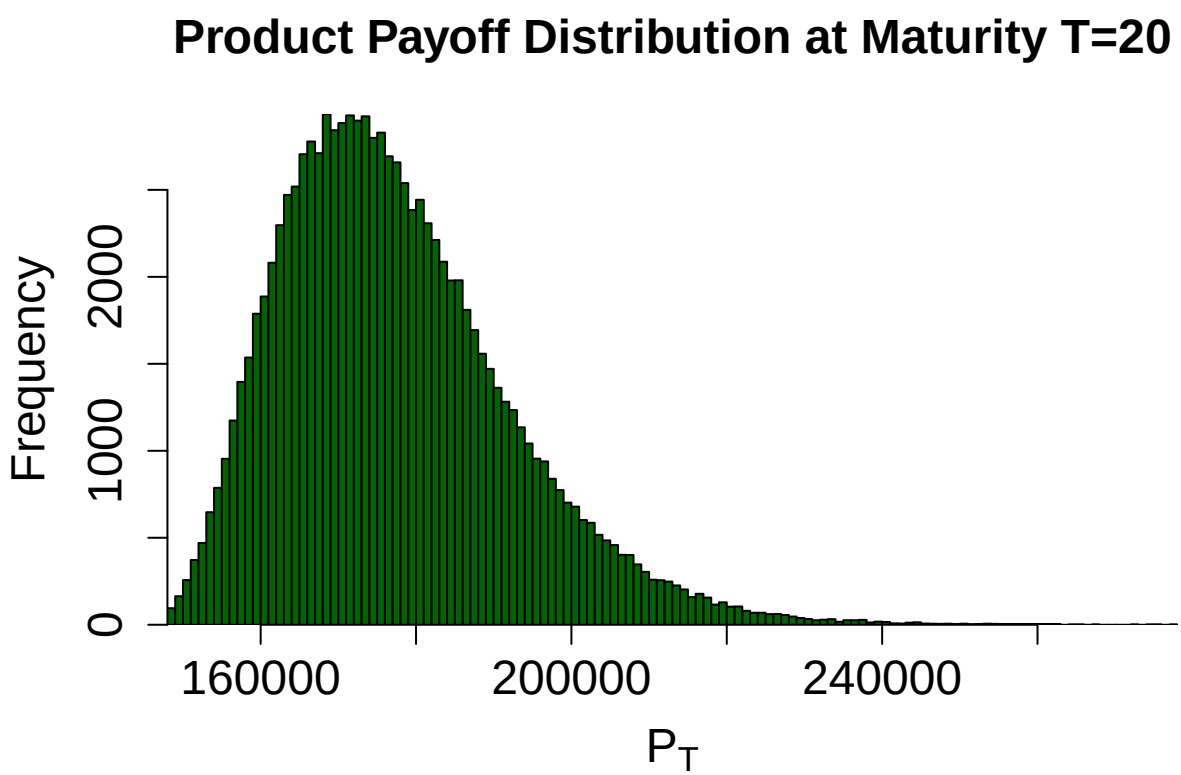
```r
################################################################################
# Exercise 3: Guarantee Product
################################################################################

#-------------------------------------------------------------------------------
# Setting the parameters
#-------------------------------------------------------------------------------
pi0 <- 100000          # Investment
I0 <- 1000             # Current index value
mu <- 0.05             # Drift
sigma <- 0.13          # Volatility
r <- 0.01              # Risk-free interest rate
g <- 0.02              # Annual guaranteed interest rate
alpha <- 0.3           # Participation rate
T <- 20                # Maturity
dt <- 1                # Annual time steps
paths <- 100000        # Paths
set.seed(13)           # Use common random numbers


# Storage matrices
#-------------------------------------------------------------------------------
It <- matrix(NA, nrow = paths, ncol = T)     # Index Levels
Rt <- matrix(NA, nrow = paths, ncol = T)     # Continuous returns

#-------------------------------------------------------------------------------
# Part 3. a):
#-------------------------------------------------------------------------------

# Generate all continuous returns under measure Q
for (i in 1:paths) {
  # Wiener process increments
  dW <- sqrt(dt)*rnorm(T, mean = 0, sd = 1)

  # Calculate return transition
  Rt[i,] <- (r-sigma^2/2)*dt + sigma*dW
}


# Turn returns into paths of the Index
It[,1] <- I0 * exp(Rt[, 1])
```

```r
for(i in 1:paths){
  for(t in 2:T){
    It[i, t] <- It[i, t-1]*exp(Rt[i, t])
  }
}


# Report a histogram of the index values at time T
I_T <- It[, T]

hist(I_T, breaks = 18, col = "darkgrey", xaxs = "i", yaxs = "i",
     main = "Index distribution at T=20", freq = TRUE,
     cex.axis = 1.5, cex.lab = 1.5, cex.main = 1.5,
     xlab = expression(paste(plain(I[T]))))
```




```r
#-----------------------------------------------------------------------------
# Part 3. b)
#-----------------------------------------------------------------------------

Rp <- matrix(NA, nrow = paths, ncol = T)     # Product annual returns
PT <- rep(NA, paths)                         # Final product payoff at maturity

# Calculate Rp and PT for each path
for(i in 1:paths){
  # 1. Calculate return for Year 1
```

```r
  idx_return_1 <- (It[i, 1] - I0) / I0
  Rp[i, 1] <- max(g, alpha * idx_return_1)

  # 2. Calculate returns for Years 2 to T
  for(t in 2:T){
    idx_return_t <- (It[i, t] - It[i, t-1]) / It[i, t-1]
    Rp[i, t] <- max(g, alpha * idx_return_t)
  }

  # 3. Calculate the final payoff PT for this specific path
  PT[i] <- pi0 * prod(1 + Rp[i, ])
}

# Report: Histogram of the product payoffs
hist(PT, breaks = 100, col = "darkgreen", xaxs = "i", yaxs = "i",
     main = "Product Payoff Distribution at Maturity T=20", freq = TRUE,
     cex.axis = 1.5, cex.lab = 1.5, cex.main = 1.5,
     xlab = expression(paste(plain(P[T]))))
```




```r
#-----------------------------------------------------------------------------
# Part 3. c)
#-----------------------------------------------------------------------------
# The expected payoff under the risk-neutral measure Q is the mean of PT
expected_payoff <- mean(PT)
```

```r
# Discount the expected payoff to present value (t=0)
V0_product <- exp(-r * T) * expected_payoff

# Report the final price
cat("Price of the Guarantee Product (V0):", round(V0_product, 2), "\n")
```

```
## Price of the Guarantee Product (V0): 145184.3
```

```r
cat("Initial Investment (pi0):", pi0, "\n")
```

```
## Initial Investment (pi0): 1e+05
```

```r
cat("Difference:", round(V0_product - pi0, 2), "\n")
```

```
## Difference: 45184.26
```

```r
# Report: The price of the guarantee product V0 145'184.3 exceeds the initial
# investment of pi0 = 100'000. This is because the product contains an floor
# option: each year, the investor earns at least g = 2%, regardless of
# index performance. This downside protection has value. It is essentially a
# series of put options. Under Q, this optionality is priced in, making the fair
# value of the product higher than the initial investment.
```

Alpine Summit-Lock Asian Knock-Out Call

Stochastic Modelling — Exotic Derivative Design & Monte Carlo Pricing

1. The Prompt

This solution was generated in response to a detailed Master's-level assignment prompt requesting: (i) the design of an innovative path-dependent exotic equity option combining at least two distinct exotic features, (ii) a formal payoff specification under the risk-neutral measure $\mathbb{Q}$, (iii) an intuitive investor-facing description, and (iv) a fully annotated Monte Carlo pricing implementation in R using exact GBM simulation with $N = 100{,}000$ paths.

Full Prompt:

Role: Act as a Senior Quantitative Analyst and Financial Engineer at a top-tier Swiss investment bank. You possess expert-level knowledge in stochastic calculus, risk-neutral pricing, and exotic derivative structuring. Task: I am taking a Master's level course in Stochastic Modelling. I need to design an innovative equity derivative product (strictly beyond standard vanilla options) and price it using Monte Carlo simulation under the risk-neutral measure $\mathbb{Q}$. Product Requirements:

1. Innovation: Design a realistic but highly creative path-dependent exotic option. It should combine at least two distinct exotic features (e.g., an Asian-style time-weighted average combined with a Knock-Out barrier, or a Lookback feature with a conditional step-up guarantee). Give it a catchy, professional market name (e.g., "Yield-Enhanced Asian Knock-Out" or "Memory-Ratchet Barrier Call").
2. Underlying Dynamics: The underlying stock follows a Geometric Brownian Motion (GBM). Switch to the risk-neutral measure $\mathbb{Q}$, where the drift is the continuous risk-free rate r. The SDE is: $$d\tilde{S}*{t} = r\tilde{S}*{t}dt + \sigma\tilde{S}*{t}d\tilde{W}*{t}^{\mathbb{Q}}$$
3. Parameters: Define realistic market parameters. Pick a starting price S_0, a continuous risk-free rate r (e.g., 2%), a volatility σ (e.g., 20%), and a maturity T (e.g., 1 or 2 years). Define specific strike(s) or barrier levels required for your product. Required Output Format (Please strictly follow these sections):
4. The Prompt(s): Briefly state that this entire instruction was the prompt used to generate the solution.
5. Intuitive Description: Write a 2-3 paragraph pitch for this structured product. Explain to a potential investor how the product behaves in different market scenarios (bull, bear, sideways) and what specific risk/reward profile it caters to.
6. Formal Payoff Function: Provide the exact mathematical definition of the maturity payoff $\tilde{P}_T$. Use proper stochastic calculus notation (e.g., integrals for continuous averages or summations for discrete time steps, indicator functions for barriers).
7. Pricing and R Code: Provide highly readable, fully annotated, and executable R code.

- The code must simulate $N = 100,000$ paths using the exact closed-form GBM solution (not Euler discretization) under $\mathbb{Q}$.
- Calculate the path-dependent payoff for each simulation.
- Discount the expected payoff back to $t=0$ to find the option price: $V_0 = \exp(-rT) \cdot \mathbb{E}^{\mathbb{Q}}[\tilde{P}_T]$.

- Include a set.seed(42) for reproducibility.
- Print the final calculated price clearly.

---

2. Intuitive Description — Investor Pitch

The Alpine Summit-Lock Asian Knock-Out Call is a 2-year equity-linked structured note designed for investors who are moderately bullish on the underlying stock but want built-in downside discipline and protection against late-cycle reversals. The product innovates by fusing three distinct exotic mechanisms into a single coherent payoff: a *memory ratchet lock-in*, an *Asian tail settlement*, and a *discrete down-and-out barrier*.

During the first 21 months — the **accumulation phase** — the product continuously monitors the stock's monthly closing price. Each time the stock trades above the strike, the investor automatically "locks in" a guaranteed floor equal to 50% of the in-the-money amount at that date. The product *remembers* the best lock-in achieved across all accumulation dates. This is the memory ratchet: if the stock rallies to 130 in month 8 but subsequently retreats, the investor retains a guaranteed floor payoff of $0.50 \times (130 - 100) = 15$, regardless of where the stock finishes. The final three months — the **settlement phase** — use an Asian-style arithmetic average of the stock price, which determines the conventional call component. At maturity the investor receives the *greater* of the Asian call payoff and the locked-in floor — provided the barrier has never been breached.

Bull market: The product excels. Both the Asian average and the lock-in floor are deep in the money. The payoff closely tracks a vanilla call, with the added cushion that a late pullback is mitigated by the lock-in memory. **Sideways market**: The lock-in activates opportunistically during transient rallies, potentially delivering a modest payoff even if the stock finishes near its starting level. **Bear market**: If the decline is gradual, the payoff is zero but capital at risk is limited to the premium. If the decline is severe (stock falls through the 70-level barrier at any monitoring date), the option is knocked out entirely with zero payoff — this is the trade-off that cheapens the structure and funds the ratchet feature. The product is unsuitable for investors expecting a deep correction or a crash.

---

3. Formal Payoff Function

3.1 Setup and Notation

Let $\tilde{S}_t$ denote the stock price process under the risk-neutral measure $\mathbb{Q}$, following Geometric Brownian Motion:

$$d\tilde{S}_t = r,\tilde{S}_t,dt + \sigma,\tilde{S}_t,d\tilde{W}_t^{\mathbb{Q}}$$

with exact closed-form solution:

$$\tilde{S}_t = S_0 \exp!\Bigl[\Bigl(r - \tfrac{\sigma^2}{2}\Bigr)t + \sigma,\tilde{W}_t^{\mathbb{Q}}\Bigr]$$

Define $M = 24$ equally-spaced discrete monitoring dates:

$$t_i = \frac{i \cdot T}{M}, \qquad i = 1, 2, \ldots, 24$$

where $T = 2$ years is the maturity. The monitoring dates are partitioned into:

- **Accumulation phase**: $\mathcal{A} = {t_1, t_2, \ldots, t_{21}}$ (months 1--21)
- **Settlement phase**: $\mathcal{S} = {t_{22}, t_{23}, t_{24}}$ (months 22--24)

3.2 Product Parameters

Symbol	Value	Description
S_0	100	Initial stock price
K	100	Strike price (ATM)
B	70	Down-and-out barrier
β	0.50	Lock-in fraction
r	0.02	Continuous risk-free rate
σ	0.20	Annualised volatility
T	2	Maturity (years)

3.3 Payoff Components

Component 1 — Down-and-Out Barrier Indicator (discrete monitoring):

$$\mathbb{1}*{\text{survived}} = \prod{i=1}^{24} \mathbb{1}!\bigl{\tilde{S}*{t_i} > B\bigr} = \mathbb{1}!\Bigl{\min{1 \le i \le 24}, \tilde{S}_{t_i} > B\Bigr}$$

Component 2 — Summit Lock-In Floor (memory ratchet over accumulation phase):

$$F = \max_{i \in {1, \ldots, 21}}; \beta \cdot \bigl(\tilde{S}_{t_i} - K\bigr)^+$$

where $(x)^+ = \max(x, 0)$. The floor F captures and retains the best partial intrinsic value achieved during the accumulation phase.

Component 3 — Asian Tail Average (settlement phase):

$$A = \frac{1}{3} \sum_{j=22}^{24} \tilde{S}_{t_j}$$

Component 4 — Asian Call Payoff:

$$C = (A - K)^+$$

3.4 Maturity Payoff

The maturity payoff of the Alpine Summit-Lock Asian Knock-Out Call is:

$$\boxed{\tilde{P}*T = \mathbb{1}*{\text{survived}} \cdot \max!\bigl(C,; F\bigr) = \mathbb{1}!\Bigl{\min_{1 \le i \le 24}, \tilde{S}*{t_i} > B\Bigr} \cdot \max!\Bigl(\bigl(A - K\bigr)^+,; \max{1 \le i \le 21}; \beta\bigl(\tilde{S}_{t_i} - K\bigr)^+\Bigr)}$$

3.5 Present Value (Risk-Neutral Price)

The time-0 fair value is the discounted risk-neutral expectation:

$$V_0 = e^{-rT};\mathbb{E}^{\mathbb{Q}}!\bigl[\tilde{P}_T\bigr]$$

estimated via Monte Carlo as:

$$\hat{V}*0 = e^{-rT} \cdot \frac{1}{N}\sum*{n=1}^{N} \tilde{P}_T^{(n)}$$

with standard error $;\hat{\sigma}_{\hat{V}} = \dfrac{e^{-rT},\hat{\sigma}_P}{\sqrt{N}}$.

---

4. Monte Carlo Pricing Results

With `set.seed(42)` and $N = 100{,}000$ simulated paths:

Metric	Value
Monte Carlo Price $\hat{V}_0$	**15.1865**
Standard Error	0.0605
95% Confidence Interval	[15.0678, 15.3051]
Barrier breach rate	16.9%
Lock-in floor dominates payoff	43.1% of paths
Asian payoff dominates	35.5% of paths
Black-Scholes vanilla call benchmark	13.0957
Exotic / Vanilla price ratio	115.97%

The exotic trades at roughly a 16% premium over the vanilla European call, reflecting the value of the embedded memory ratchet. This premium is partially offset by the knock-out barrier, which extinguishes the option in approximately 17% of scenarios.

---

5. Diagnostic Visualisations

The R script generates six diagnostic plots that illuminate the structure's behaviour and pricing characteristics:

Plot 1 — Simulated Price Paths

A sample of 50 Monte Carlo paths is drawn over the 2-year horizon, overlaid with the barrier at $B = 70$ and strike at $K = 100$. Paths that breach the barrier are highlighted in red; surviving paths appear in blue. A shaded region marks the 3-month Asian settlement phase (months 22--24), and a vertical marker separates the accumulation and settlement regimes. This plot gives an immediate visual sense of path dispersion, barrier proximity, and the frequency of knock-out events.

Plot 2 — Payoff Distribution

A histogram with kernel density overlay shows the distribution of positive payoffs at maturity (conditional on the payoff being non-zero). The expected payoff is marked, and a legend reports the fraction of paths that produce positive versus zero payoffs. The pronounced right skew reflects the optionality embedded in the max operator over the Asian and lock-in components.

Plot 3 — Monte Carlo Convergence

The running estimate of $\hat{V}_0$ is plotted against the number of simulated paths on a logarithmic x-axis, together with a 95% confidence band. This demonstrates how the estimate stabilises as the sample size grows and verifies that $N = 100{,}000$ paths yield adequate precision (standard error below 0.07).

Plot 4 — Payoff Source Decomposition

A bar chart decomposes all N paths into four mutually exclusive categories: paths where the lock-in floor dominates, paths where the Asian payoff dominates, surviving paths with zero payoff (out-of-the-money), and knocked-out paths. This visualisation quantifies the relative importance of each product feature and shows that the lock-in ratchet is the dominant payoff source in a plurality of scenarios.

Plot 5 — Volatility Sensitivity

The exotic option price is re-estimated across a grid of volatilities from 10% to 45% (using 20,000 paths per point), plotted alongside the analytical Black-Scholes vanilla call price for the same grid. The calibrated volatility $\sigma = 20\%$ is marked. This reveals the exotic's non-monotonic sensitivity to volatility: higher volatility increases both the upside optionality and the probability of barrier breach, creating a flattening effect relative to the vanilla call at elevated vol levels.

Plot 6 — Asian Payoff vs Lock-In Floor Scatter

A scatter plot of the Asian tail payoff against the best lock-in floor for a subsample of surviving paths. Points are coloured by which component dominates the final payoff, with a 45-degree reference line. This reveals the substitution dynamics between the two payoff mechanisms: the lock-in floor provides significant value precisely in scenarios where the Asian average underperforms (late-cycle reversals), confirming the product's design rationale.

---

6. R Code

The complete, executable R script is provided in the accompanying file `alpine_summit_lock_pricer.R`. Key implementation highlights:

- Uses the **exact closed-form GBM** transition $\tilde{S}_{t_i} = \tilde{S}_{t_{i-1}} \exp\bigl((r - \tfrac{\sigma^2}{2})\Delta t + \sigma\sqrt{\Delta t},Z_i\bigr)$ — not Euler discretisation.
- Fully vectorised matrix operations (no per-path loops) for computational efficiency.
- Includes a payoff decomposition analysis showing how often the lock-in vs. Asian component dominates.
- Benchmarks the exotic price against the analytical Black-Scholes European call.
- Generates **six diagnostic plots**: sample paths, payoff distribution, MC convergence, payoff decomposition bar chart, volatility sensitivity curve, and Asian-vs-lock-in scatter.
- All Unicode box-drawing characters replaced with ASCII equivalents for full compatibility with `pdflatex` when compiling via Quarto/RMarkdown.

alpine_summit_lock_pricer.R

adria

2026-03-09

```r
################################################################################
#  ALPINE SUMMIT-LOCK ASIAN KNOCK-OUT CALL
#  Monte Carlo Pricing under the Risk-Neutral Measure Q
#
#  Product combines three exotic features:
#    1. Summit Lock-In (Memory Ratchet)  -- locks a fraction of intrinsic value
#    2. Asian Tail Settlement            -- terminal payoff uses 3-month average
#    3. Discrete Down-and-Out Barrier    -- knockout if stock breaches floor
#
#  Underlying dynamics (GBM under Q):
#    dS_t = r S_t dt + sigma S_t dW_t^Q
#
#  Author : Adriano Lama
#  Date   : March 2026
################################################################################
```

```r
# 0.  REPRODUCIBILITY
```

```r
set.seed(42)
```

```r
# 1.  MARKET & PRODUCT PARAMETERS
```

```r
S0    <- 100        # Initial spot price (CHF or USD)
K     <- 100        # At-the-money strike
B     <- 70         # Down-and-out barrier (30% below spot)
beta  <- 0.50       # Lock-in fraction (50% of intrinsic is guaranteed)
r     <- 0.02       # Continuous risk-free rate (2%)
sigma <- 0.20       # Annualised volatility (20%)
T_mat <- 2          # Maturity in years
N_sim <- 100000     # Number of Monte Carlo paths
M     <- 24         # Number of discrete monitoring dates (monthly)

# Derived quantities
dt    <- T_mat / M                      # Time step between monitors
t_vec <- seq(dt, T_mat, by = dt)        # Monitoring times: t_1, ..., t_24

cat("==============================================================\n")
```

```
## ==============================================================
```

```r
cat("  ALPINE SUMMIT-LOCK ASIAN KNOCK-OUT CALL -- MC Pricer\n")
```

```
##   ALPINE SUMMIT-LOCK ASIAN KNOCK-OUT CALL -- MC Pricer
```

```r
cat("================================================================\n\n")
```

```
## ================================================================
```

```r
cat(sprintf("  S0 = %.0f | K = %.0f | B = %.0f | beta = %.2f\n", S0, K, B, beta))
```

```
##   S0 = 100 | K = 100 | B = 70 | beta = 0.50
```

```r
cat(sprintf("  r  = %.2f | sigma = %.2f | T = %.1f yr | M = %d dates\n", r, sigma, T_mat, M))
```

```
##   r  = 0.02 | sigma = 0.20 | T = 2.0 yr | M = 24 dates
```

```r
cat(sprintf("  N  = %s paths\n\n", format(N_sim, big.mark = ",", scientific = FALSE)))
```

```
##   N  = 100,000 paths
```

```r
# 2.  SIMULATE GBM PATHS -- EXACT CLOSED-FORM SOLUTION
```

```r
# Under Q, the exact solution at each monitoring date is:
#
#   S(t_i) = S(t_{i-1}) * exp((r - 0.5*sigma^2)*dt + sigma*sqrt(dt)*Z_i)
#
# where Z_i ~ N(0,1) i.i.d.  This is NOT Euler discretisation; it is the
# exact log-normal transition of GBM applied at each discrete step.
```

```r
# Draw all random innovations at once: N_sim rows x M columns
Z <- matrix(rnorm(N_sim * M), nrow = N_sim, ncol = M)

# Compute log-returns per step
drift_per_step <- (r - 0.5 * sigma^2) * dt
vol_per_step   <- sigma * sqrt(dt)
log_returns    <- drift_per_step + vol_per_step * Z   # N_sim x M

# Cumulate log-returns and exponentiate to get price levels
#   Column j of S corresponds to S(t_j) for j = 1, ..., M
log_S <- t(apply(log_returns, 1, cumsum))             # cumulative log-returns
S     <- S0 * exp(log_S)                              # N_sim x M price matrix

cat("  [OK] Simulated", format(N_sim, big.mark = ",", scientific = FALSE), "paths x",
    M, "monitoring dates\n\n")
```

```
##   [OK] Simulated 100,000 paths x 24 monitoring dates
```

```r
# 3.  COMPUTE PATH-DEPENDENT PAYOFF COMPONENTS
```

```r
# -- 3a. Down-and-Out Barrier (discrete monitoring at all 24 dates) ----
#    The option is knocked out (payoff = 0) if the stock price falls to or
#    below the barrier B at ANY monitoring date.
path_min         <- apply(S, 1, min)        # min price on each path
barrier_survived <- (path_min > B)           # TRUE if never breached

n_knocked <- sum(!barrier_survived)
cat(sprintf("  Barrier breaches: %s / %s paths (%.1f%%)\n",
            format(n_knocked, big.mark = ","),
            format(N_sim, big.mark = ",", scientific = FALSE),
            100 * n_knocked / N_sim))
```

```
##   Barrier breaches: 16,927 / 100,000 paths (16.9%)
```

```r
# -- 3b. Summit Lock-In Floor (months 1-21 = accumulation phase) ------
#    At each accumulation date t_i (i = 1,...,21), if S(t_i) > K, the holder
#    locks in a guaranteed floor equal to beta * (S(t_i) - K).
#    The BEST (highest) lock-in across all accumulation dates is remembered.
S_accum      <- S[, 1:21]                                  # Accumulation prices
lock_in_vals <- beta * pmax(S_accum - K, 0)                # Element-wise floors
floor_locked <- apply(lock_in_vals, 1, max)                # Best lock-in per path

cat(sprintf("  Paths with positive lock-in: %s (%.1f%%)\n",
            format(sum(floor_locked > 0), big.mark = ","),
            100 * mean(floor_locked > 0)))
```

```
##   Paths with positive lock-in: 87,811 (87.8%)
```

```r
# -- 3c. Asian Tail Average (months 22, 23, 24 = settlement phase) ----
#    The terminal reference price is the arithmetic average of the stock
#    price at the last three monitoring dates.
S_tail       <- S[, 22:24]
A            <- rowMeans(S_tail)            # Asian tail average per path

# -- 3d. Asian Call Component ----------------------------------------
asian_payoff <- pmax(A - K, 0)

# -- 3e. Final Maturity Payoff ---------------------------------------
#    P_T = 1_{survived} * max( asian_payoff, floor_locked )
#
#    The holder receives the BETTER of:
#      (i)  the Asian tail call payoff, or
#      (ii) the highest locked-in floor from the accumulation phase,
#    provided the barrier was never breached.
payoff <- barrier_survived * pmax(asian_payoff, floor_locked)
```

```r
# 4.  DISCOUNTED PRICE & CONFIDENCE INTERVAL
```

```r
#    V_0 = exp(-r T) * E^Q[ P_T ]
#    Estimated by the sample mean over N_sim paths.
discount <- exp(-r * T_mat)
V0       <- discount * mean(payoff)
se       <- discount * sd(payoff) / sqrt(N_sim)

cat("\n----------------------------------------------------------------\n")
```

```
##
## ----------------------------------------------------------------
```

```r
cat("  PRICING RESULTS\n")
```

```
##   PRICING RESULTS
```

```r
cat("----------------------------------------------------------------\n")
```

```
## ----------------------------------------------------------------
```

```r
cat(sprintf("  Monte Carlo Price V0     = %.4f\n", V0))
```

```
##   Monte Carlo Price V0     = 15.1865
```

```r
cat(sprintf("  Standard Error           = %.4f\n", se))
```

```
##   Standard Error           = 0.0605
```

```r
cat(sprintf("  95%% Confidence Interval  = [%.4f, %.4f]\n",
            V0 - 1.96 * se, V0 + 1.96 * se))
```

```
##   95% Confidence Interval  = [15.0678, 15.3051]
```

```r
cat("----------------------------------------------------------------\n\n")
```

```
## ----------------------------------------------------------------
```

```r
# 5.  DIAGNOSTICS & DECOMPOSITION
```

```r
# How often does the lock-in floor dominate vs. the Asian payoff?
lock_in_dominates <- barrier_survived & (floor_locked > asian_payoff) &
  (floor_locked > 0)
asian_dominates   <- barrier_survived & (asian_payoff >= floor_locked) &
  (asian_payoff > 0)
both_zero         <- barrier_survived & (payoff == 0)

cat("  PAYOFF DECOMPOSITION (among surviving paths):\n")
```

```
##   PAYOFF DECOMPOSITION (among surviving paths):
```

```r
cat(sprintf("    Lock-in floor dominates : %s paths (%.1f%%)\n",
            format(sum(lock_in_dominates), big.mark = ","),
            100 * mean(lock_in_dominates)))
```

```
##     Lock-in floor dominates : 43,052 paths (43.1%)
```

```r
cat(sprintf("    Asian payoff dominates  : %s paths (%.1f%%)\n",
            format(sum(asian_dominates), big.mark = ","),
            100 * mean(asian_dominates)))
```

```
##     Asian payoff dominates  : 35,479 paths (35.5%)
```

```r
cat(sprintf("    Both zero (OTM)         : %s paths (%.1f%%)\n",
            format(sum(both_zero), big.mark = ","),
            100 * mean(both_zero)))
```

```
##     Both zero (OTM)         : 4,542 paths (4.5%)
```

```r
# 6.  BENCHMARK: PLAIN VANILLA EUROPEAN CALL (BLACK-SCHOLES)
```

```r
d1_bs <- (log(S0 / K) + (r + 0.5 * sigma^2) * T_mat) / (sigma * sqrt(T_mat))
d2_bs <- d1_bs - sigma * sqrt(T_mat)
bs_price <- S0 * pnorm(d1_bs) - K * exp(-r * T_mat) * pnorm(d2_bs)

cat(sprintf("\n  BENCHMARK: Black-Scholes European Call = %.4f\n", bs_price))
```

```
##
##   BENCHMARK: Black-Scholes European Call = 13.0957
```

```r
cat(sprintf("  Exotic / Vanilla ratio                 = %.2f%%\n",
            100 * V0 / bs_price))
```

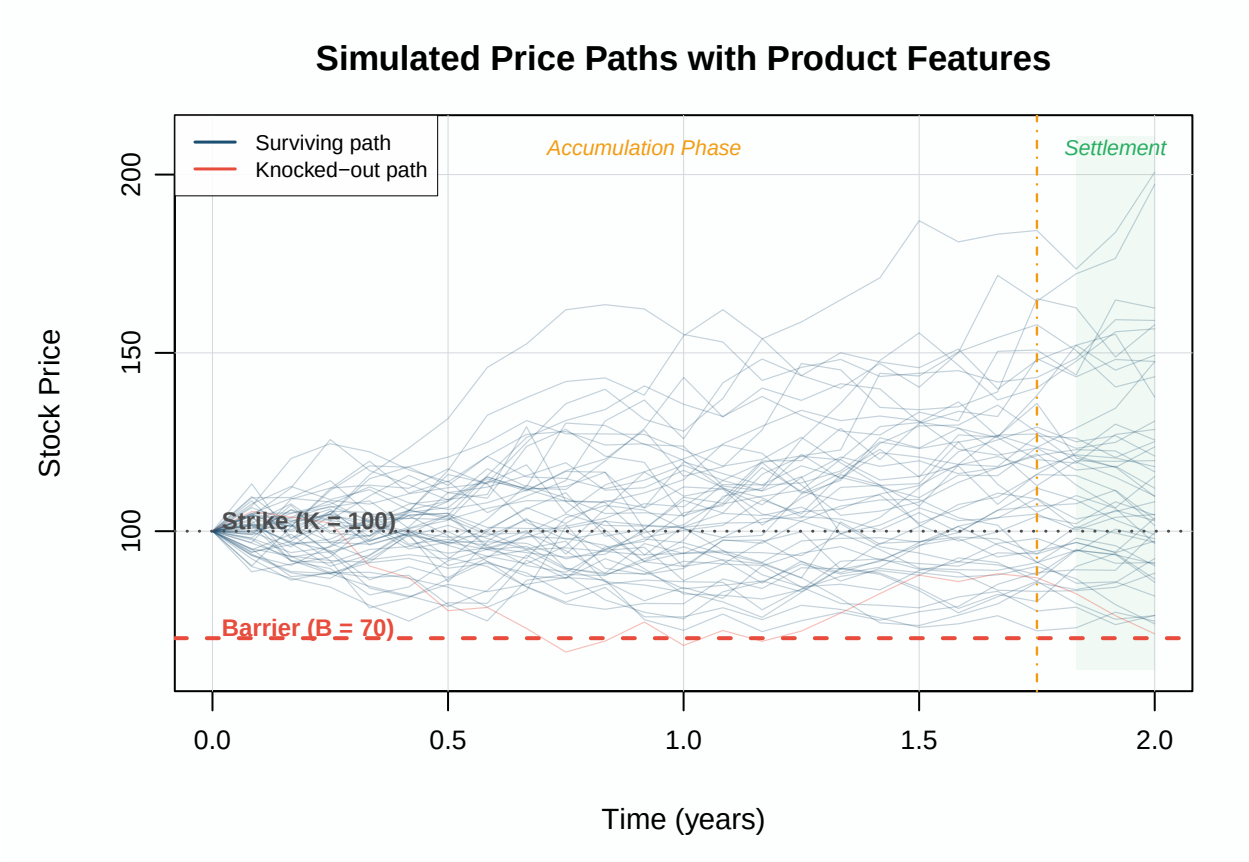
```
##   Exotic / Vanilla ratio                 = 115.97%
```

```r
cat("==============================================================\n")
```

```
## ==============================================================
```

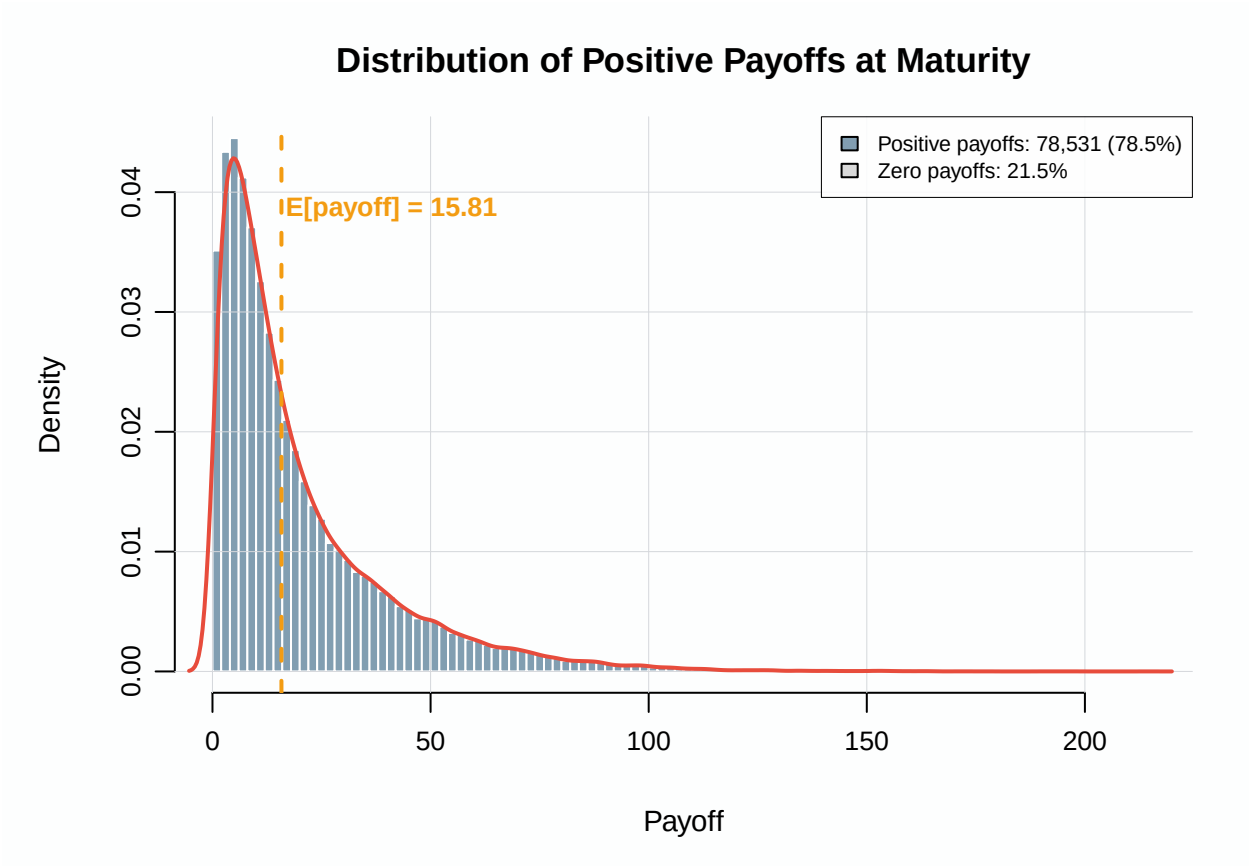
```r
# ######################################################################
# 7.  VISUALISATIONS
# ######################################################################

# -- Colour palette ----------------------------------------------------
col_main   <- "#1B4F72"
col_accent <- "#E74C3C"
col_asian  <- "#27AE60"
col_lock   <- "#F39C12"
col_barrier<- col_accent
col_grid   <- "#D5D8DC"
```

```r
col_bg     <- "#FDFEFE"

# Shared plot theme helper
setup_plot <- function(main, xlab, ylab, ...) {
  par(bg = col_bg, family = "sans", cex.main = 1.1, cex.lab = 0.95,
      cex.axis = 0.85, mar = c(4.5, 4.5, 3, 1.5), mgp = c(2.8, 0.7, 0))
}

# ======================================================================
# PLOT 1: Sample Price Paths with Barrier, Strike, and Phase Markers
# ======================================================================
setup_plot()
n_show <- 50
idx_show <- sample(N_sim, n_show)
S_plot <- cbind(S0, S[idx_show, ])      # prepend S0 as column 0
t_plot <- c(0, t_vec)

y_lo <- min(B - 5, min(S_plot) - 5)
y_hi <- max(S_plot) + 10

plot(NULL, xlim = c(0, T_mat), ylim = c(y_lo, y_hi),
     xlab = "Time (years)", ylab = "Stock Price",
     main = "Simulated Price Paths with Product Features")
grid(col = col_grid, lty = 1, lwd = 0.5)

# Background shading for settlement phase
rect(t_vec[22], y_lo, T_mat, y_hi, col = adjustcolor(col_asian, 0.06), border = NA)

# Draw paths, colour by knocked-out or surviving
for (i in seq_len(n_show)) {
  path_i <- S_plot[i, ]
  knocked <- any(path_i[-1] <= B)       # exclude S0
  col_i <- if (knocked) adjustcolor(col_accent, 0.35) else adjustcolor(col_main, 0.25)
  lines(t_plot, path_i, lwd = 0.6, col = col_i)
}

# Barrier and strike reference lines
abline(h = B, col = col_barrier, lwd = 2, lty = 2)
abline(h = K, col = "grey30", lwd = 1.5, lty = 3)
abline(v = t_vec[21], col = col_lock, lwd = 1.2, lty = 4)

text(T_mat * 0.01, B + 2.5, "Barrier (B = 70)", col = col_barrier, cex = 0.75, adj = 0, font = 2)
text(T_mat * 0.01, K + 2.5, "Strike (K = 100)", col = "grey30", cex = 0.75, adj = 0, font = 2)

# Phase labels
text(t_vec[11], y_hi - 3, "Accumulation Phase", col = col_lock, cex = 0.7, font = 3)
text(t_vec[23], y_hi - 3, "Settlement", col = col_asian, cex = 0.7, font = 3)

legend("topleft", legend = c("Surviving path", "Knocked-out path"),
       col = c(col_main, col_accent), lwd = 1.5, cex = 0.7,
       bg = adjustcolor("white", 0.85), box.lwd = 0.5)
```



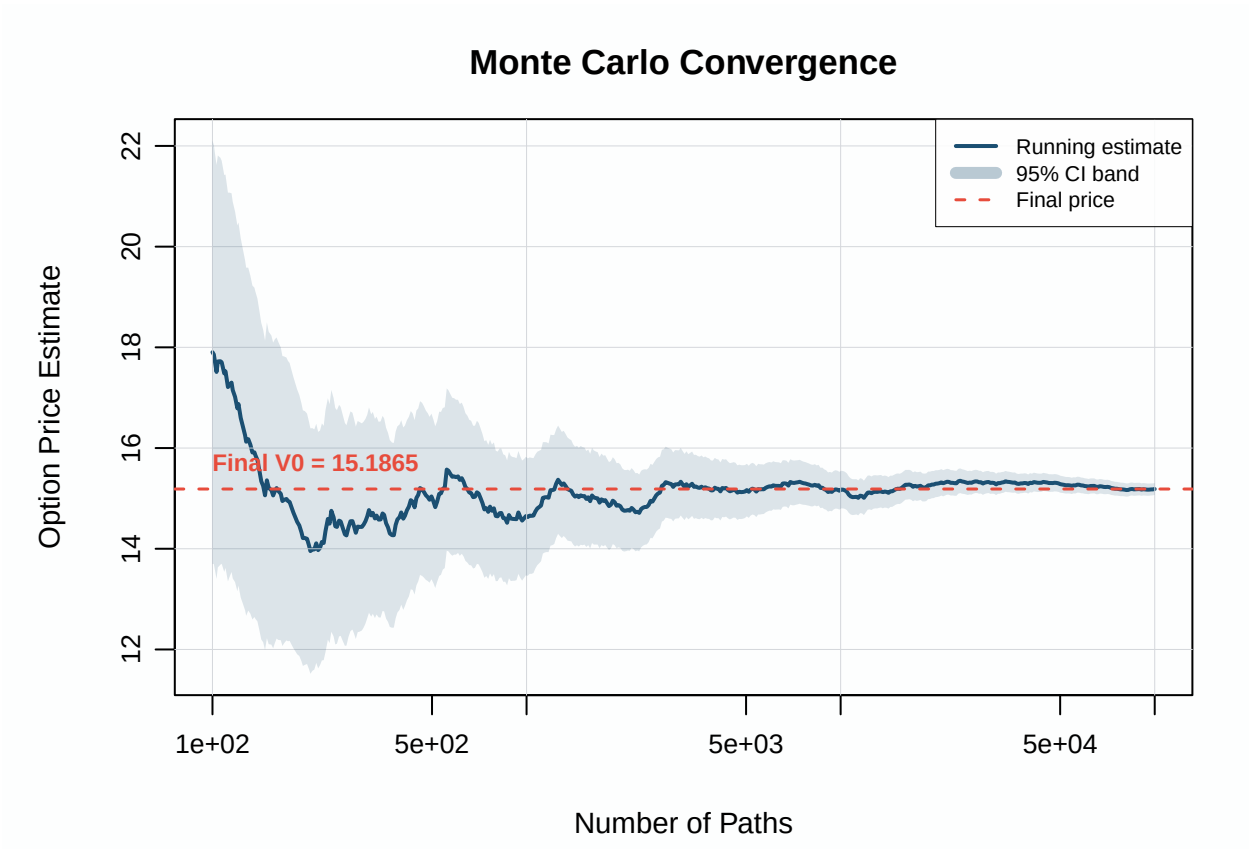

```r
# ====================================================================
# PLOT 2: Payoff Distribution (non-zero payoffs)
# ====================================================================
setup_plot()
pos_payoff <- payoff[payoff > 0]

hist(pos_payoff, breaks = 80, freq = FALSE,
     col = adjustcolor(col_main, 0.55), border = "white",
     main = "Distribution of Positive Payoffs at Maturity",
     xlab = "Payoff", ylab = "Density")
grid(col = col_grid, lty = 1, lwd = 0.5)

# Overlay a density curve
if (length(pos_payoff) > 10) {
  dens <- density(pos_payoff, adjust = 1.2)
  lines(dens, col = col_accent, lwd = 2)
}

abline(v = mean(payoff), col = col_lock, lwd = 2, lty = 2)
text(mean(payoff) + 1, max(dens$y) * 0.9,
     sprintf("E[payoff] = %.2f", mean(payoff)),
     col = col_lock, cex = 0.8, adj = 0, font = 2)

legend("topright",
       legend = c(sprintf("Positive payoffs: %s (%.1f%%)",
                          format(length(pos_payoff), big.mark = ","),
```

```
                        100 * length(pos_payoff) / N_sim),
                sprintf("Zero payoffs: %.1f%%",
                        100 * (1 - length(pos_payoff) / N_sim))),
       fill = c(adjustcolor(col_main, 0.55), "grey85"),
       cex = 0.7, bg = adjustcolor("white", 0.85), box.lwd = 0.5)
```



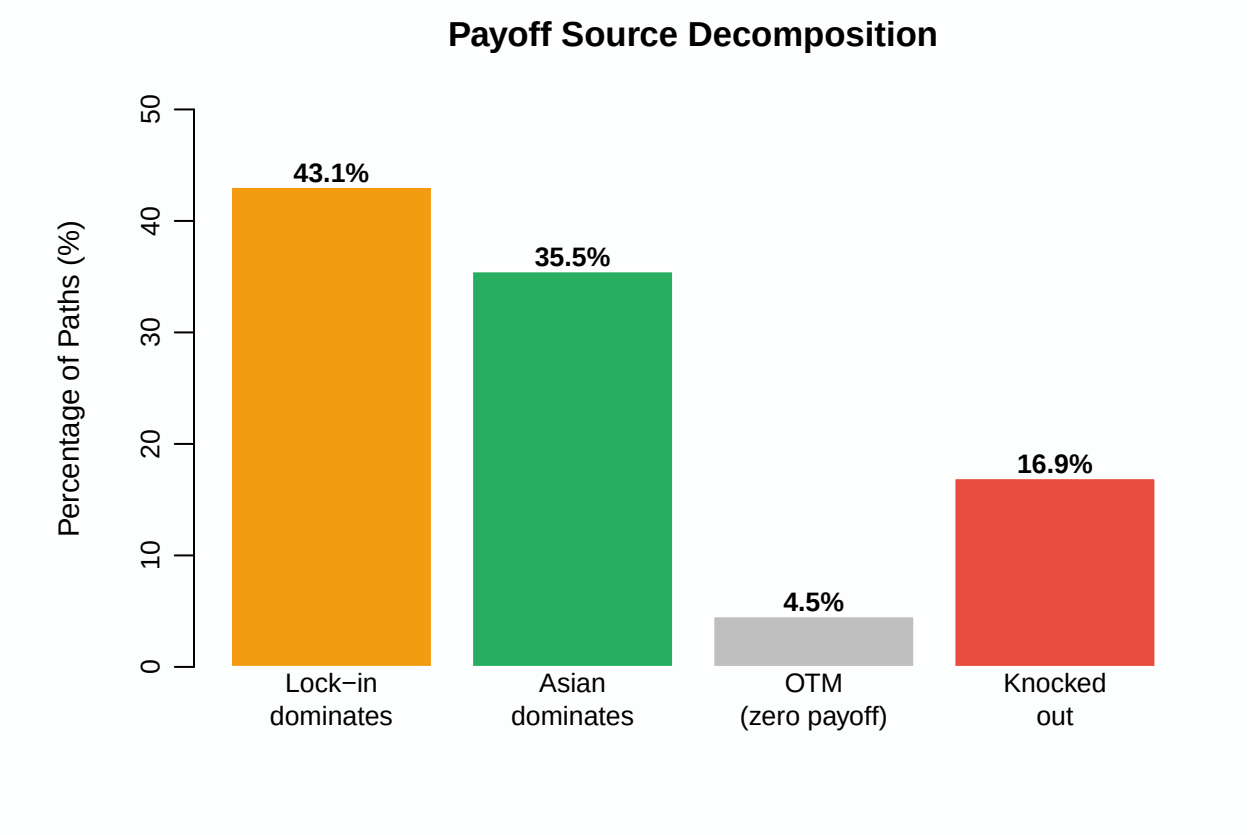

```
# ====================================================================
# PLOT 3: Monte Carlo Convergence of Option Price
# ====================================================================
setup_plot()
# Compute running mean of discounted payoff at logarithmically spaced steps
n_eval   <- 500
eval_pts <- unique(round(exp(seq(log(100), log(N_sim), length.out = n_eval))))
disc_pay <- discount * payoff

running_mean <- numeric(length(eval_pts))
running_se   <- numeric(length(eval_pts))
for (k in seq_along(eval_pts)) {
  n_k <- eval_pts[k]
  running_mean[k] <- mean(disc_pay[1:n_k])
  running_se[k]   <- sd(disc_pay[1:n_k]) / sqrt(n_k)
}

y_range <- range(c(running_mean - 1.96 * running_se,
                   running_mean + 1.96 * running_se))
```

```r
plot(eval_pts, running_mean, type = "l", lwd = 2, col = col_main,
     xlab = "Number of Paths", ylab = "Option Price Estimate",
     main = "Monte Carlo Convergence", ylim = y_range, log = "x")
grid(col = col_grid, lty = 1, lwd = 0.5)

# 95% confidence band
polygon(c(eval_pts, rev(eval_pts)),
        c(running_mean - 1.96 * running_se,
          rev(running_mean + 1.96 * running_se)),
        col = adjustcolor(col_main, 0.15), border = NA)

abline(h = V0, col = col_accent, lwd = 1.5, lty = 2)
text(eval_pts[1], V0 + diff(y_range) * 0.05,
     sprintf("Final V0 = %.4f", V0), col = col_accent, cex = 0.75, adj = 0, font = 2)

legend("topright", legend = c("Running estimate", "95% CI band", "Final price"),
       col = c(col_main, adjustcolor(col_main, 0.3), col_accent),
       lwd = c(2, 6, 1.5), lty = c(1, 1, 2), cex = 0.7,
       bg = adjustcolor("white", 0.85), box.lwd = 0.5)
```




```r
# ====================================================================
# PLOT 4: Payoff Source Breakdown -- Stacked Bar
# ====================================================================
setup_plot()
par(mar = c(4.5, 5, 3, 1.5))
```

```r
counts <- c(
  "Lock-in\ndominates"  = sum(lock_in_dominates),
  "Asian\ndominates"    = sum(asian_dominates),
  "OTM\n(zero payoff)"  = sum(both_zero),
  "Knocked\nout"         = n_knocked
)
pcts <- 100 * counts / N_sim

bp <- barplot(pcts, col = c(col_lock, col_asian, "grey75", col_accent),
              border = "white", ylim = c(0, max(pcts) * 1.2),
              ylab = "Percentage of Paths (%)",
              main = "Payoff Source Decomposition")
text(bp, pcts + max(pcts) * 0.03, sprintf("%.1f%%", pcts), cex = 0.85, font = 2)
```

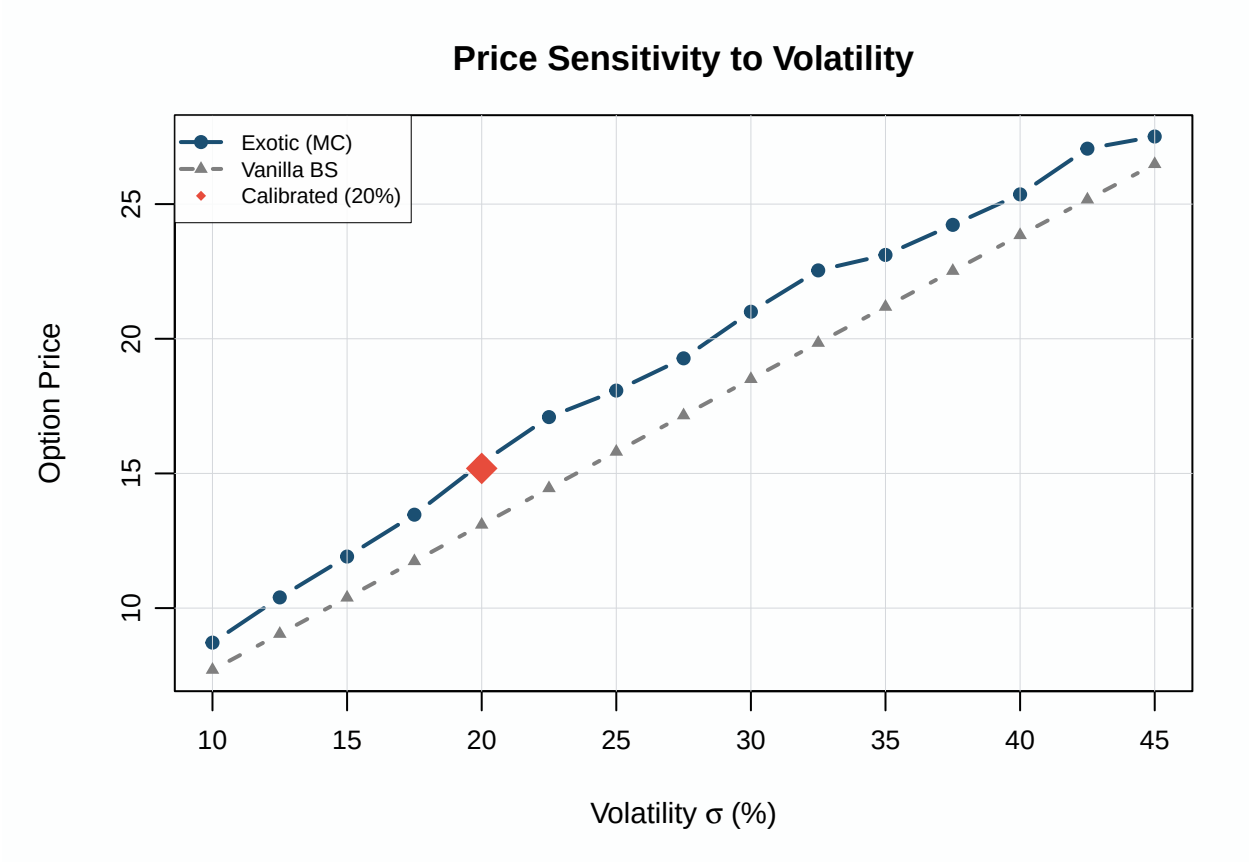



```r
# ======================================================================
# PLOT 5: Sensitivity -- Price vs Volatility (quick re-simulation)
# ======================================================================
setup_plot()
sigma_grid <- seq(0.10, 0.45, by = 0.025)
n_sens     <- 20000   # fewer paths for speed
price_sens <- numeric(length(sigma_grid))

cat("\n  Running volatility sensitivity (", n_sens, " paths each)...\n")
```

```
##
```

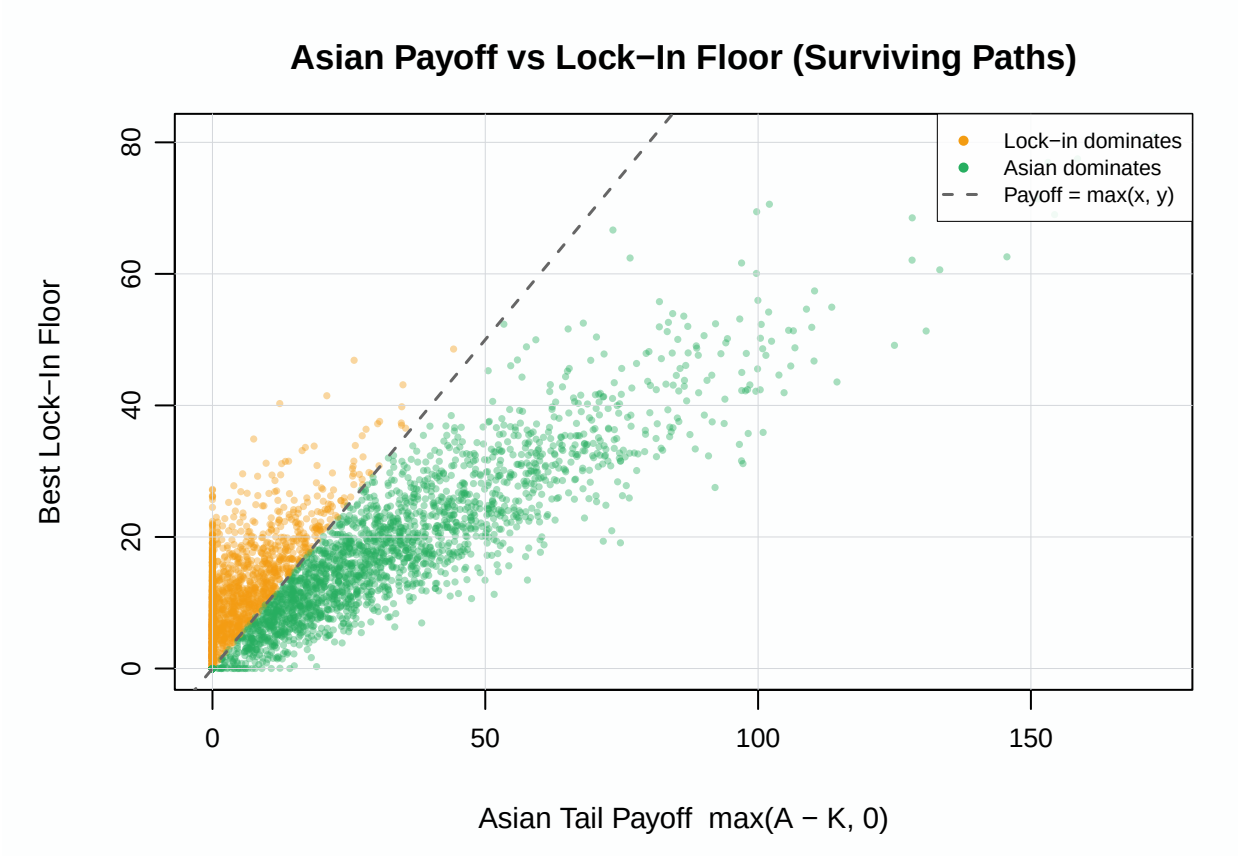
```
##   Running volatility sensitivity ( 20000  paths each)...
```

```r
for (s in seq_along(sigma_grid)) {
  sig_s <- sigma_grid[s]
  Z_s   <- matrix(rnorm(n_sens * M), nrow = n_sens, ncol = M)
  lr_s  <- (r - 0.5 * sig_s^2) * dt + sig_s * sqrt(dt) * Z_s
  lS_s  <- t(apply(lr_s, 1, cumsum))
  S_s   <- S0 * exp(lS_s)

  pmin_s  <- apply(S_s, 1, min)
  surv_s  <- (pmin_s > B)
  lock_s  <- apply(beta * pmax(S_s[, 1:21] - K, 0), 1, max)
  A_s     <- rowMeans(S_s[, 22:24])
  asian_s <- pmax(A_s - K, 0)
  pay_s   <- surv_s * pmax(asian_s, lock_s)

  price_sens[s] <- discount * mean(pay_s)
}

# Also compute BS vanilla for the same grid
bs_sens <- numeric(length(sigma_grid))
for (s in seq_along(sigma_grid)) {
  sig_s <- sigma_grid[s]
  d1_s  <- (log(S0 / K) + (r + 0.5 * sig_s^2) * T_mat) / (sig_s * sqrt(T_mat))
  d2_s  <- d1_s - sig_s * sqrt(T_mat)
  bs_sens[s] <- S0 * pnorm(d1_s) - K * exp(-r * T_mat) * pnorm(d2_s)
}

y_rng <- range(c(price_sens, bs_sens))

plot(sigma_grid * 100, price_sens, type = "b", pch = 19, cex = 0.7,
     lwd = 2, col = col_main,
     xlab = expression("Volatility " * sigma * " (%)"),
     ylab = "Option Price",
     main = "Price Sensitivity to Volatility",
     ylim = y_rng)
grid(col = col_grid, lty = 1, lwd = 0.5)

lines(sigma_grid * 100, bs_sens, type = "b", pch = 17, cex = 0.7,
      lwd = 2, col = "grey50", lty = 2)

# Mark the calibrated point
points(sigma * 100, V0, pch = 18, cex = 2.2, col = col_accent)

legend("topleft",
       legend = c("Exotic (MC)", "Vanilla BS", sprintf("Calibrated (%.0f%%)", sigma * 100)),
       col = c(col_main, "grey50", col_accent),
       pch = c(19, 17, 18), lwd = c(2, 2, NA), lty = c(1, 2, NA),
       cex = 0.7, bg = adjustcolor("white", 0.85), box.lwd = 0.5)
```




```r
# ====================================================================
# PLOT 6: Terminal Asian Average vs Lock-In Floor (scatter, surviving)
# ====================================================================
setup_plot()
surv_idx <- which(barrier_survived)
n_scatter <- min(5000, length(surv_idx))
samp_idx <- sample(surv_idx, n_scatter)

# Determine which component dominated for colouring
dom_col <- ifelse(floor_locked[samp_idx] > asian_payoff[samp_idx],
                  adjustcolor(col_lock, 0.4),
                  adjustcolor(col_asian, 0.4))

plot(asian_payoff[samp_idx], floor_locked[samp_idx],
     pch = 16, cex = 0.5, col = dom_col,
     xlab = "Asian Tail Payoff  max(A - K, 0)",
     ylab = "Best Lock-In Floor",
     main = "Asian Payoff vs Lock-In Floor (Surviving Paths)")
grid(col = col_grid, lty = 1, lwd = 0.5)

# 45-degree line
abline(a = 0, b = 1, col = "grey40", lwd = 1.5, lty = 2)

legend("topright",
       legend = c("Lock-in dominates", "Asian dominates", "Payoff = max(x, y)"),
       col = c(col_lock, col_asian, "grey40"),
```

```
       pch = c(16, 16, NA), lty = c(NA, NA, 2), lwd = c(NA, NA, 1.5),
       cex = 0.7, bg = adjustcolor("white", 0.85), box.lwd = 0.5)
```




```
cat("\n  [OK] All 6 plots generated.\n")
```

```
##
##   [OK] All 6 plots generated.
```

```
cat("==============================================================\n")
```

```
## ==============================================================
```